

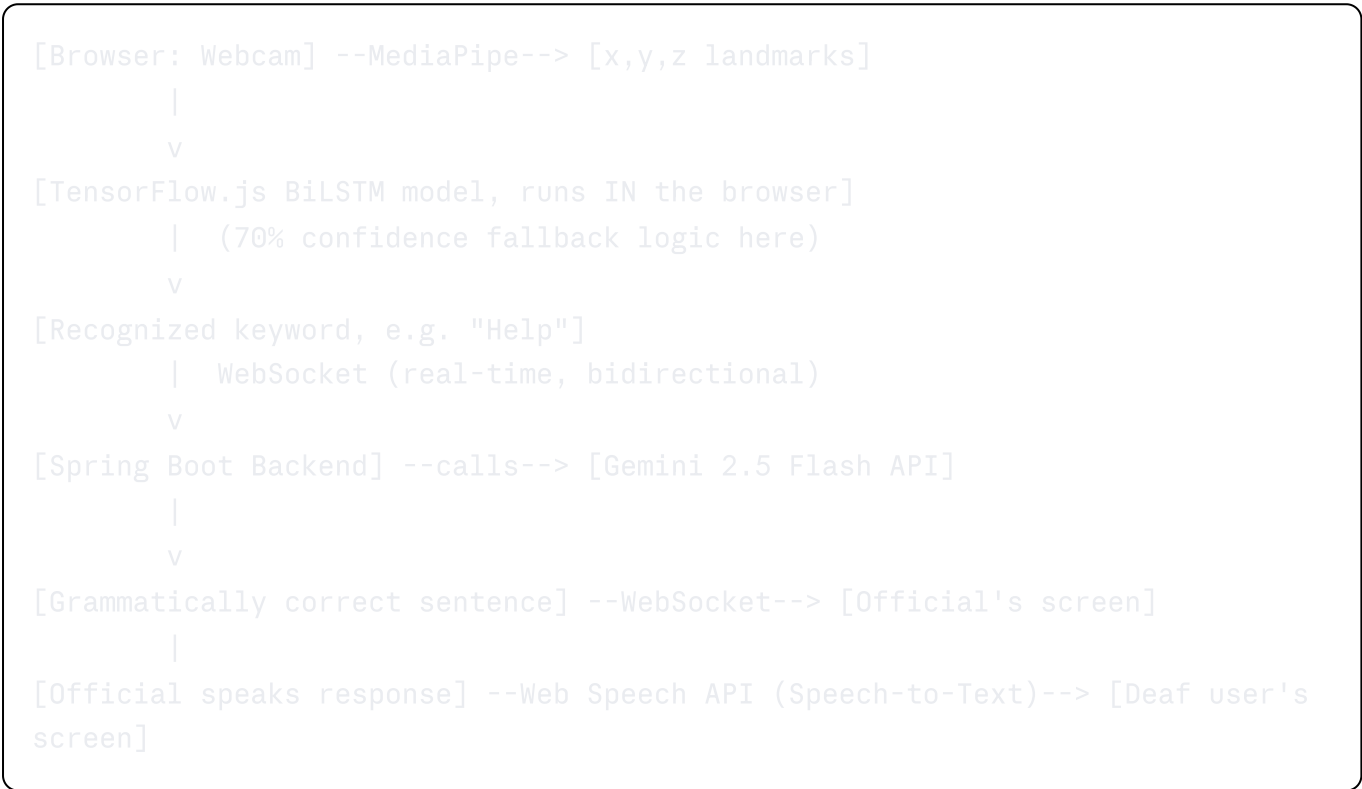
Implementation Roadmap

AI-Based Interpretation and Translation System for Indian Sign Language

Team: Jeneesh Joshi, Shreya Jadhav, Neelakshi Kar, Ashmeet Sharma | **Guide:** Dr. Garima Joshi

This is a from-zero, step-by-step build guide matching the architecture already defined in your Research Foundation Dossier: **MediaPipe → BiLSTM (TensorFlow.js, in-browser) → Spring Boot WebSocket → Gemini 2.5 Flash → Speech-to-Text feedback loop.**

0. The Big Picture



Four layers, four teammates — this maps naturally onto your 4-person team:

1. **Data Acquisition + Frontend** (webcam, MediaPipe, UI)
2. **Intelligent Processing** (BiLSTM training, TensorFlow.js)
3. **Contextual Generation** (Spring Boot WebSocket + Gemini API)
4. **Bidirectional Feedback** (Speech-to-Text) + integration/testing

Split ownership this way from day one so nobody is blocked waiting on someone else's layer — each layer can be built and tested in isolation before wiring them together.

1. Tools & Environment Setup (Do this first, Week 1)

Install on every team member's machine:

Tool	Purpose	Notes
Node.js (LTS) + npm	Frontend, TensorFlow.js, WebSocket client	<code>node -v</code> to confirm
Python 3.10+	Model training (BiLSTM)	Use a virtual env: <code>python -m venv isl-env</code>
VS Code	Shared editor	Install ESLint, Python extensions
Java 17+ & Maven/Gradle	Spring Boot backend	<code>java -version</code>
Git + GitHub repo	Version control	One repo, branch-per-feature
Google AI Studio account	Gemini 2.5 Flash API key	Free tier is enough for prototyping
Webcam-equipped laptop	Data collection	Consistent lighting matters a lot

Python libraries (for training):

```
bash
pip install mediapipe opencv-python numpy pandas scikit-learn tensorflow matplotlib
```

Repo structure to create immediately:

```
isl-translator/
├── data-collection/      # Python scripts to record landmark sequences
│   └── dataset/         # CSV/npz files per sign, per team member
├── model-training/      # Jupyter notebooks + training scripts
│   └── saved_model/     # Keras model before conversion
├── frontend/           # HTML/JS + TensorFlow.js inference + MediaPipe
│   └── model/           # Converted model.json + weights
├── backend/            # Spring Boot project (WebSocket + Gemini proxy)
└── docs/               # SAD, KPIs, meeting notes
```

Set this up in Week 1 and get a "hello world" running in each folder (a blank webcam page, a blank Spring Boot app that starts, a Python script that imports mediapipe successfully) before writing any real logic. This catches environment problems early instead of on demo day.

2. Phase 1 — Data Acquisition Layer (Weeks 1-2)

Goal: Reliably extract (x, y, z) skeletal landmarks from a live webcam feed.

Step-by-step

1. **Start with MediaPipe's JavaScript solution** (Hands + Pose, or Holistic if you want face/pose too — your dossier mentions MediaPipe Holistic for richer accuracy). Load it via a simple HTML page first, not React — you want to isolate this layer.
2. Get the webcam stream into a `<video>` element using `navigator.mediaDevices.getUserMedia`.
3. Feed each video frame into MediaPipe's landmark detector. It returns 21 points per hand (x, y, z each) — for two hands that's up to 126 numbers per frame.
4. **Draw the landmarks on a `<canvas>` overlay** so you can visually confirm tracking is working (this is your sanity check — do this before anything else).
5. Package each frame's landmarks into a flat array/vector. This vector is what feeds the model later.
6. **Critical for your privacy claim (H1 in your hypothesis):** never send the raw video frame anywhere. Only the numeric landmark vector should ever leave this layer. Confirm this architecturally now — it's much harder to retrofit later, and it's a core claim in your dossier.

Deliverable for Week 2: A webpage that shows your webcam feed with hand skeleton overlaid, and logs a clean array of landmark coordinates to the console for every frame, at a stable frame rate (aim for 20-30 FPS in-browser).

3. Phase 2 — Dataset Creation (Weeks 2-3)

Your dossier specifies a **phased scope**: start with 3 words ("Hello", "Help", "Emergency"), prove the pipeline, then scale to 40+.

Step-by-step

1. **Build a "recording mode"** into your Phase 1 webpage: press a key to start/stop capturing a sequence of landmark frames for one sign (a sign is a short video, not a single frame — this is why BiLSTM matters, it needs a *sequence*).
2. For each sign, record **~30-50 sequences per person**, each sequence being ~20-40 frames long (roughly 1-2 seconds of the gesture). With a 4-person team doing this for 3 signs, you'll have a reasonably sized, diverse dataset quickly.
3. **Vary conditions deliberately:** different lighting, slightly different hand positions/speed, different people. This is what will make your model generalize — the literature review you already did (Table in Section 5) flags "sensitive to environmental perturbations" as a common failure; deliberately combat that here.
4. Save each sequence as a row in a CSV or as a `.npy` array: `[frame_1_landmarks, frame_2_landmarks, ..., label]`.
5. Also record a "no gesture" / resting-hand class — your dossier explicitly calls out "differentiating intentional gestures from resting hand movements" as a linguistic

challenge. Without a negative class, your model can't do this.

Deliverable for Week 3: A labeled dataset folder with balanced examples of "Hello", "Help", "Emergency", and "No Gesture", each as a fixed-length sequence of landmark vectors.

4. Phase 3 — Intelligent Processing Layer: BiLSTM (Weeks 4-6)

Goal: Train a model that classifies a sequence of landmarks into one of your sign classes.

Step-by-step

1. **Preprocess:** Pad/truncate all sequences to the same length (e.g., 30 frames).
Normalize coordinates (subtract wrist landmark position so the model is translation-invariant — this matters a lot for generalization).
2. **Split data:** train/validation/test (e.g., 70/15/15), stratified by class.
3. **Build the model in Keras:**

python

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Bidirectional, LSTM, Dense, Dropout

model = Sequential([
    Bidirectional(LSTM(64, return_sequences=True), input_shape=(30, num_features)),
    Dropout(0.3),
    Bidirectional(LSTM(32)),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dense(num_classes, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

4. **Train**, watching validation accuracy — with a small 3-word dataset expect this to converge fast. Use early stopping to avoid overfitting on your small dataset.
5. **Evaluate** using a confusion matrix — this becomes evidence for your "Recognition Accuracy" KPI.
6. **This is where your 70% Confidence Fallback logic lives:** after `model.predict()`, check `max(softmax_output)`. If it's below 0.70, don't commit to a prediction — flag it as "low confidence" so the frontend can trigger fallback (finger-spelling mode, per your Hypothesis H4).

Deliverable for Week 6: A trained `.h5`/SavedModel that classifies your 3-word dataset with a documented accuracy figure and confusion matrix — your first real KPI result.

5. Phase 4 — Bringing the Model to the Browser (Week 6)

Step-by-step

1. Install `tensorflowjs`:

bash

```
pip install tensorflowjs
```

2. Convert your trained model:

bash

```
tensorflowjs_converter --input_format=keras \
    model-training/saved_model/model.h5 \
    frontend/model/
```

This produces `model.json` + weight shard files. 3. In your frontend JS, load it:

javascript

```
const model = await tf.loadLayersModel('model/model.json');
```

4. **Wire it to Phase 1:** buffer the last 30 frames of landmarks into a rolling window, convert to a tensor, run `model.predict()` on it continuously (e.g., every few frames via `requestAnimationFrame` or a fixed interval), apply the 0.70 confidence threshold, and display the predicted word live on screen.
5. **Measure latency here** — time from "frame captured" to "prediction displayed." This is your Translation Latency KPI (target $\leq 50\text{ms}$ per your Hypothesis H2). If you're missing that target, the fixes are usually: smaller model, fewer landmarks (hands only vs. full holistic), or a shorter rolling window.

Deliverable: A live, in-browser demo — webcam in, predicted word out, entirely client-side, with a measured latency number.

6. Phase 5 — Contextual Generation Layer: Spring Boot + Gemini (Week 7)

Goal: Take recognized keywords and turn them into grammatically correct sentences, transmitted in real time.

Step-by-step

1. **Scaffold a Spring Boot project** (via start.spring.io) with dependencies: `Spring Web`, `WebSocket`.
2. **Set up a WebSocket endpoint** (`@ServerEndpoint` or Spring's `WebSocketHandler`) that the frontend connects to. This replaces plain HTTP so keywords can stream in real time without polling.

3. **Frontend side:** when your BiLSTM predicts a keyword above the confidence threshold, send it over the open WebSocket connection instead of just displaying it locally.
4. **Backend side:** on receiving a keyword (or short buffer of keywords, e.g. "Help", "Bank Account"), call the Gemini 2.5 Flash API server-side (never expose your API key in frontend JS — this is also a security/legal-feasibility point from your dossier).
 - Prompt design matters here: give Gemini a system instruction like *"You are converting isolated sign-language keywords into a single, natural, polite sentence appropriate for a banking/transit service context. Keywords: {keywords}. Output only the sentence."*
 - This is literally your **H3 hypothesis** (generative semantic synthesis) — log both the raw keyword and the generated sentence so you can later measure the accuracy/fluency improvement.
5. **Send the generated sentence back** over the same WebSocket connection to the official's screen.

Deliverable for Week 7: Typing/signing "Help" → "Emergency" produces a full sentence like *"I need urgent assistance, please help me immediately."* appearing on a second screen within a couple of seconds — your Context Generation Time KPI.

7. Phase 6 — Bidirectional Feedback: Speech-to-Text (Week 8)

Step-by-step

1. Use the browser's built-in **Web Speech API** (`SpeechRecognition`) on the official's side — no extra backend needed for a prototype.
2. When the official speaks, transcribe it live and push the transcribed text back over the same WebSocket to the deaf user's screen.
3. Optionally, feed this transcript through Gemini too, to simplify/clarify jargon-heavy official responses — this is a nice extension if time allows, not required for your baseline hypothesis.
4. Measure and log the Bidirectional Response Time KPI.

Deliverable: A genuinely two-way conversation loop — sign → sentence → official reads and speaks → deaf user reads the reply.

8. Phase 7 — Integration, Fallback, and Stress Testing (Week 9)

1. **Wire all layers together** into one cohesive demo flow, ideally with two browser windows/devices (one "deaf user" view, one "official" view) sharing a WebSocket session.
2. **Implement the fallback state machine (H4):** when confidence drops below 70% repeatedly (e.g., 3 frames in a row), switch the UI into a finger-spelling/manual character mode, let the user spell out the word, then automatically return to gesture-tracking mode. This directly answers one of your Measurable Sub-Questions.

3. **Stress test:** drop the WebSocket connection mid-conversation, simulate poor lighting, test with an out-of-vocabulary gesture — confirm the system degrades gracefully rather than freezing (this maps to your "Rigid Failure States" root-cause finding).
 4. **Log every KPI automatically** as you test: Recognition Accuracy, Translation Latency, Context Generation Time, WebSocket Reliability (successful vs dropped packets), Bidirectional Response Time. You'll want a spreadsheet of these runs for your evaluation section.
-

9. Phase 8 — Scaling and Final Polish (Week 10)

1. **Scale the dataset** from 3 words to 40+ signs (per your Feasibility Analysis schedule) — this mostly means repeating Phase 2's recording process at volume, ideally splitting sign-recording duties across all 4 teammates in parallel.
 2. **Retrain the BiLSTM** on the larger dataset; re-measure Recognition Accuracy and Word Error Rate against the 3-word baseline — this comparison is valuable evidence for your evaluation section.
 3. **UI polish:** clean, high-contrast interface (important for accessibility — practice what you preach), clear confidence/fallback indicators, simple onboarding instructions.
 4. **Prepare the baseline comparison demo:** literally show the "manual typing" baseline side-by-side with your system, timed, to make your latency/speed claims concrete for panelists (this directly operationalizes your Section 9 baseline comparison table).
 5. **Write up results against your KPIs and hypotheses (H1-H5)** — you already have the metrics defined in Section 10 of your dossier; now you're filling them with real numbers.
-

10. Suggested Week-by-Week Summary

Week	Focus	Owner(s) suggestion
1	Environment setup, repo structure, MediaPipe "hello world"	All
2	Stable landmark extraction + recording mode	Data/Frontend person
3	Record & label 3-word dataset	All (parallel recording)
4-5	Preprocess data, build & train BiLSTM	ML person
6	TensorFlow.js conversion, in-browser inference, latency measurement	ML + Frontend
7	Spring Boot WebSocket + Gemini integration	Backend person
8	Speech-to-Text bidirectional layer	Frontend/Backend
9	Full integration, fallback logic, stress testing, KPI logging	All
10	Scale to 40+ signs, retrain, polish, results write-up, demo rehearsal	All

11. Common Pitfalls to Avoid (from your own Appendix C)

- **Don't** try to record all 40+ signs before your pipeline even works end-to-end with 3 — prove the architecture first.
- **Don't** let the video frame itself leave the browser at any point — it undermines your core privacy claim.
- **Don't** skip the "no gesture" negative class — without it your model will hallucinate signs from idle hand movement.
- **Don't** hardcode your Gemini API key in frontend JavaScript — always route it through the Spring Boot backend.
- **Do** log timestamps at every layer boundary from day one (capture → prediction → WebSocket send → Gemini response → display) — this is what turns into your KPI graphs later, and it's painful to reconstruct retroactively.

This roadmap tracks directly against the Research Foundation Dossier's Sections 9–15 (Baseline, KPIs, Methodology, Feasibility, Novelty) so your implementation evidence maps cleanly back onto your existing academic writeup.